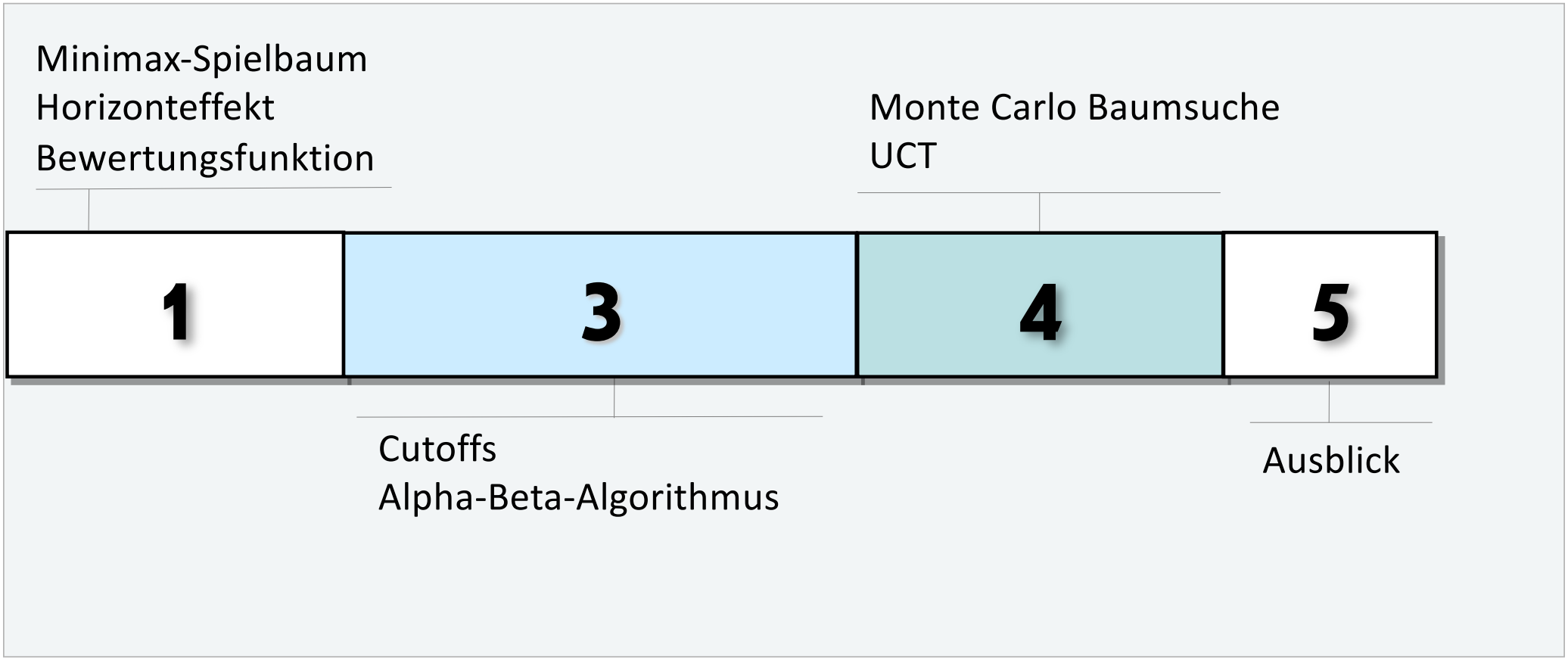
Strategische Spiele mit perfekter Information

```
1.  gameOver := false;   initialize board
2.  while not gameOver
3.    wait for legal user move
4.    make move
5.    if (mate or stalemate)  gameOver := true
6.    else
7.        search move with allocated time
8.        make move
9.        if (mate or stalemate)  gameOver := true
```

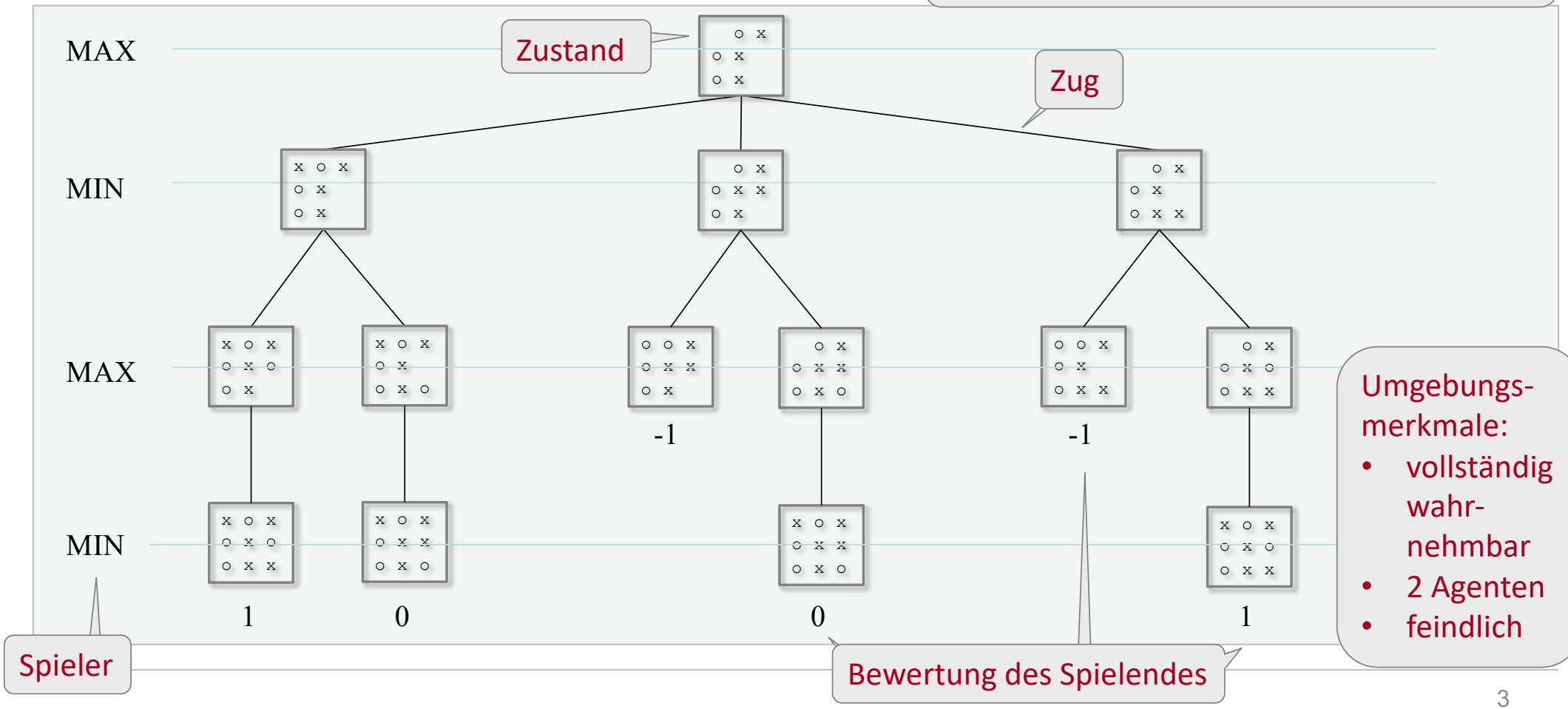


Agenda strategische Spiele



Spielbaum für Nullsummenspiel Tic Tac Toe

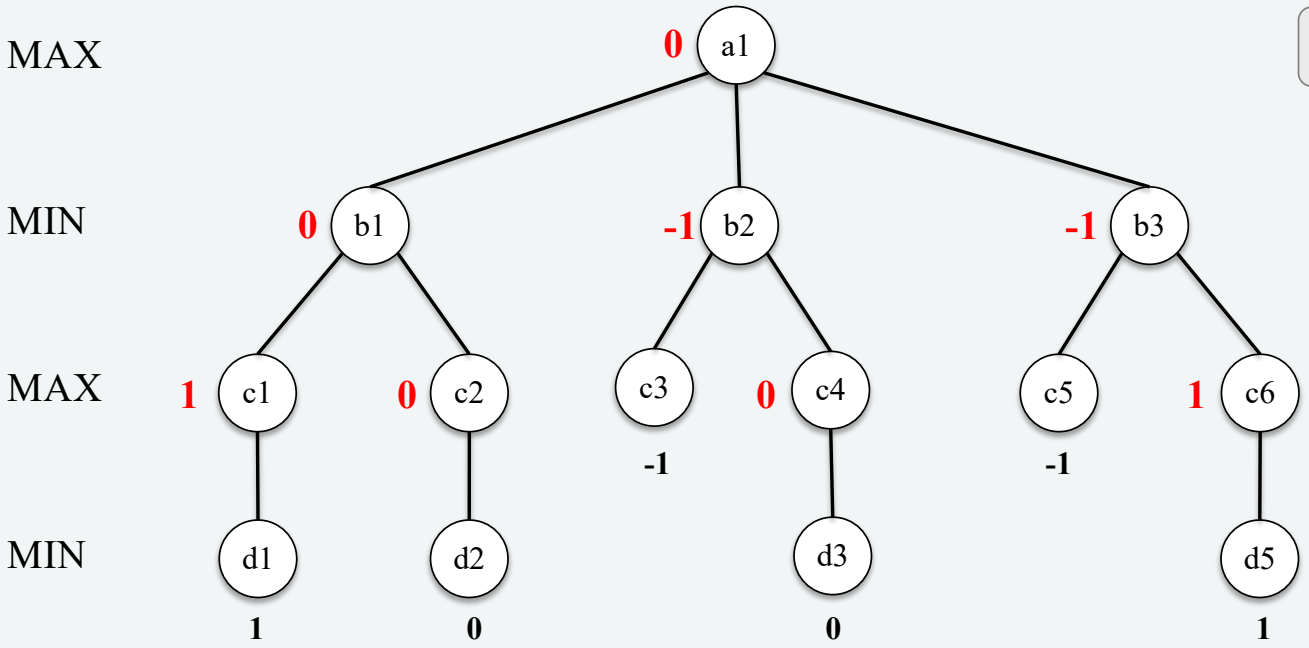
Jede Spielerin kontrolliert eine Baumebene, gute Züge wichtiger als schnelles Ende



Umgebungs-merkmale:

- vollständig wahr-nehmbar
- 2 Agenten
- feindlich

Minimax = Bottom-Up-Propagierung der besten Zustandsbewertungen

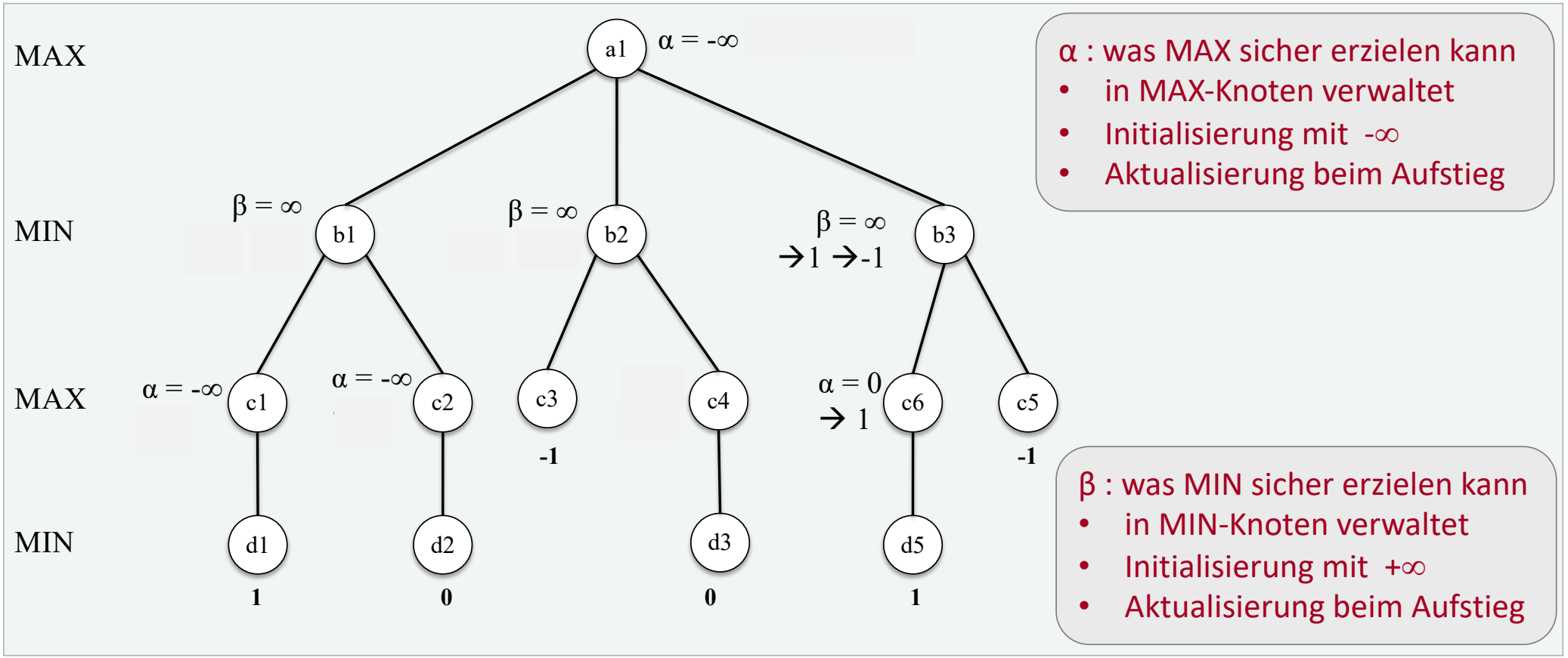


Rationalität = besten Zug wählen

Bottum-up:

- MAX wählt maximal bewerteten Nachfolgezustand
- MIN wählt minimal bewerteten Nachfolgezustand

Minimax mittels Tiefensuche unter Verwendung von α und β



α : was MAX sicher erzielen kann
- in MAX-Knoten verwaltet
- Initialisierung mit -∞
- Aktualisierung beim Aufstieg

β : was MIN sicher erzielen kann
- in MIN-Knoten verwaltet
- Initialisierung mit +∞
- Aktualisierung beim Aufstieg

Minimax ist vollständig für endliche Spielbäume

Vollständigkeit [1)]	Ja
Optimalität [2)]	Ja
Zeitkomplexität	$O(b^m)$
Speicherkomplexität	$O(b \cdot m)$

1) für endliche Spielbäume

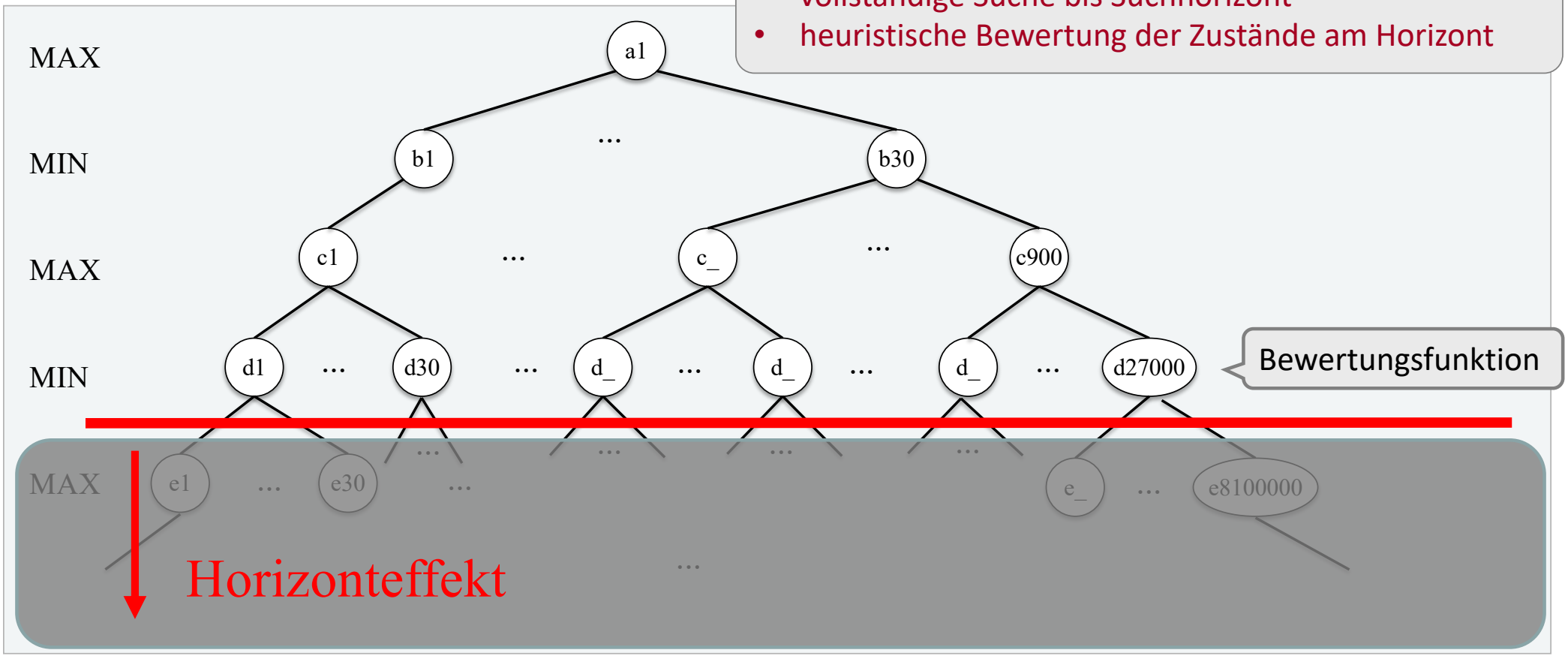
2) bestes Ergebnis nur bei perfekt rationalem Gegner

b : Verzweigungsgrad

m : maximale Baumtiefe

Spielbaumkomplexität erzwingt tiefenlimitierte Suche

- vollständige Suche bis Suchhorizont
- heuristische Bewertung der Zustände am Horizont



Kurzsichtigkeit birgt das Risiko einer falschen Stellungsbewertung

Heuristik: bewerte Materialdifferenz:
B=1, T=5, D=8, K=100

Weiß am Zug

Stellungsbewertung: -5

nach 1. Bg7-g8 D:

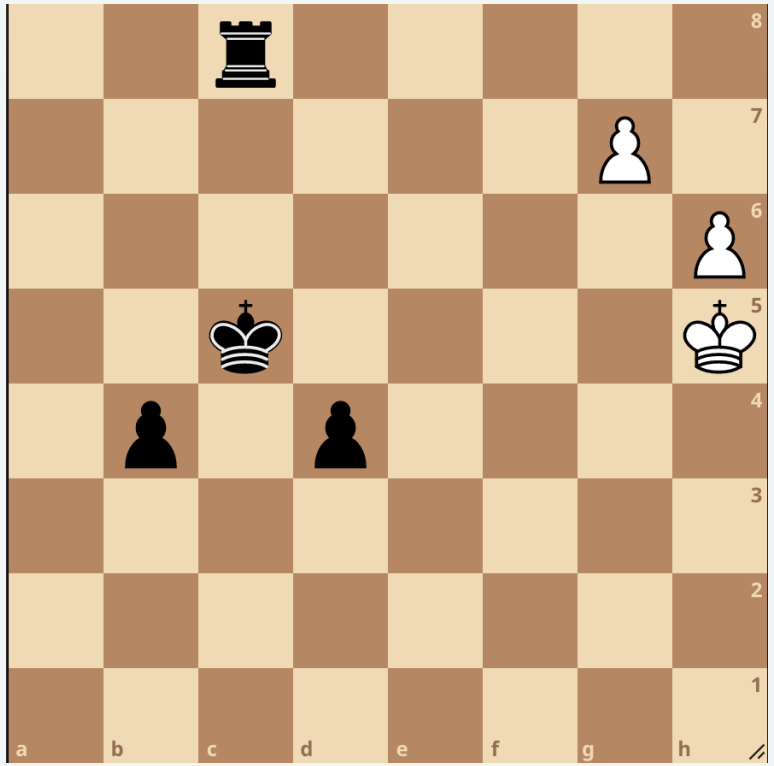
Stellungsbewertung: +2

alle anderen Züge:

Stellungsbewertung: -5

Schwarz nach 1. ... Tc8:g8

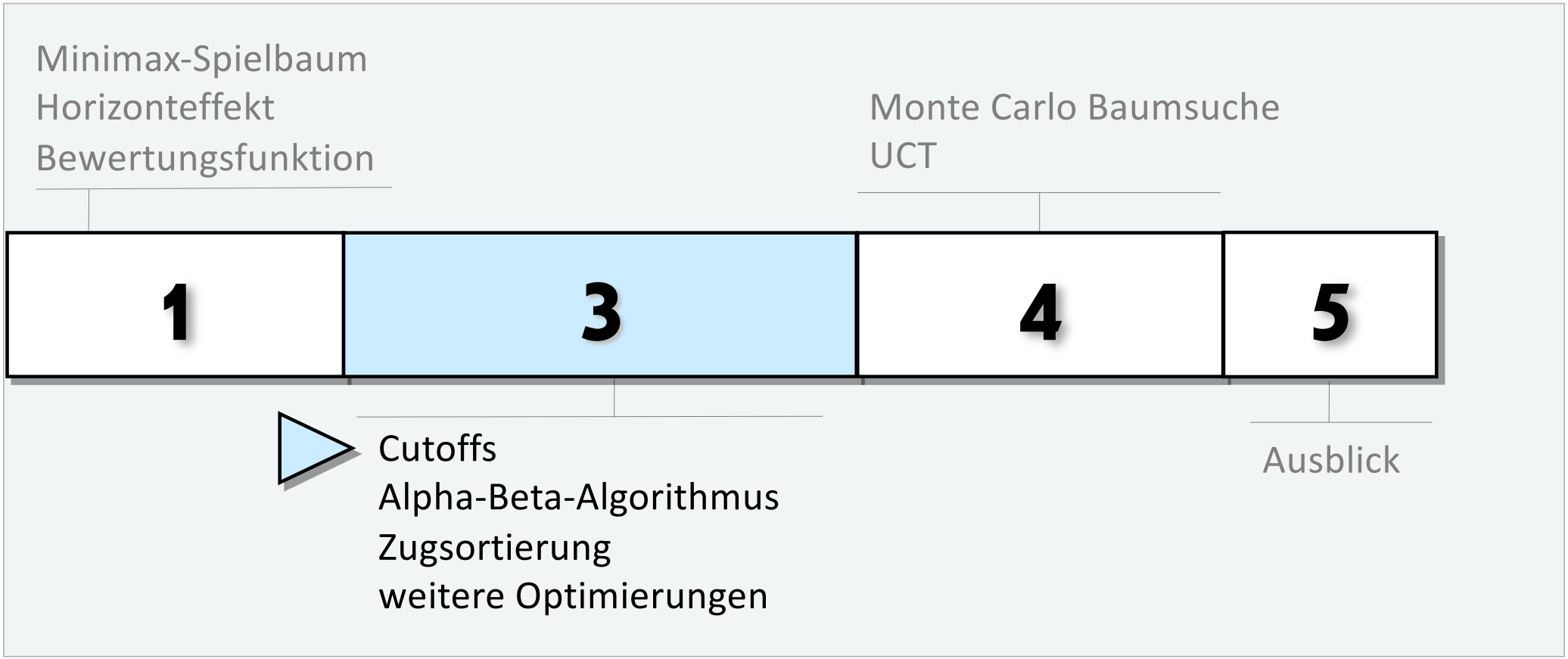
Stellungsbewertung: -6



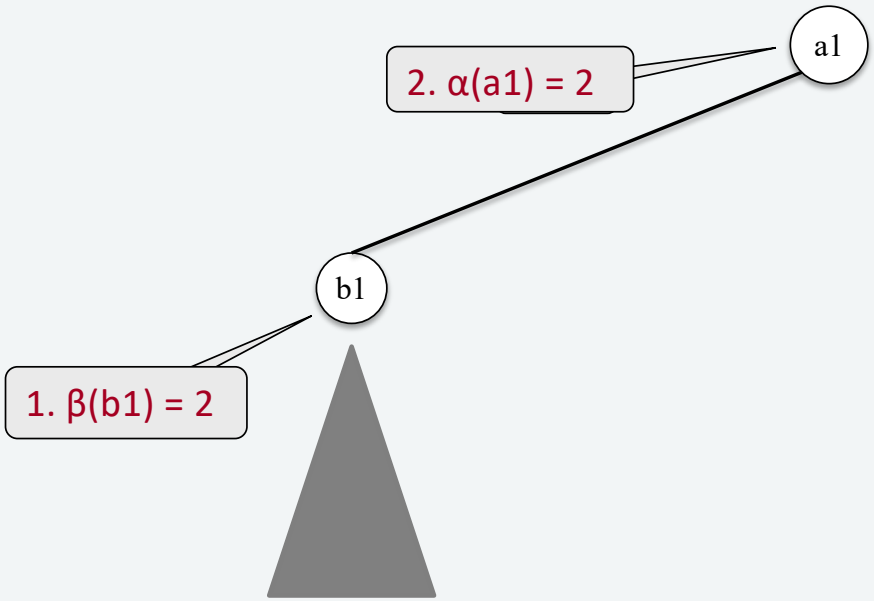
Legende: B~Bauer; T~Turm; D~Dame

https://lichess.org/analysis/fromPosition/2r5/6P1/7P/2k4K/1p1p4/8/8/8

Den Spielbaum beschneiden ohne Optimalität zu gefährden



Irrationale Züge müssen nicht untersucht werden

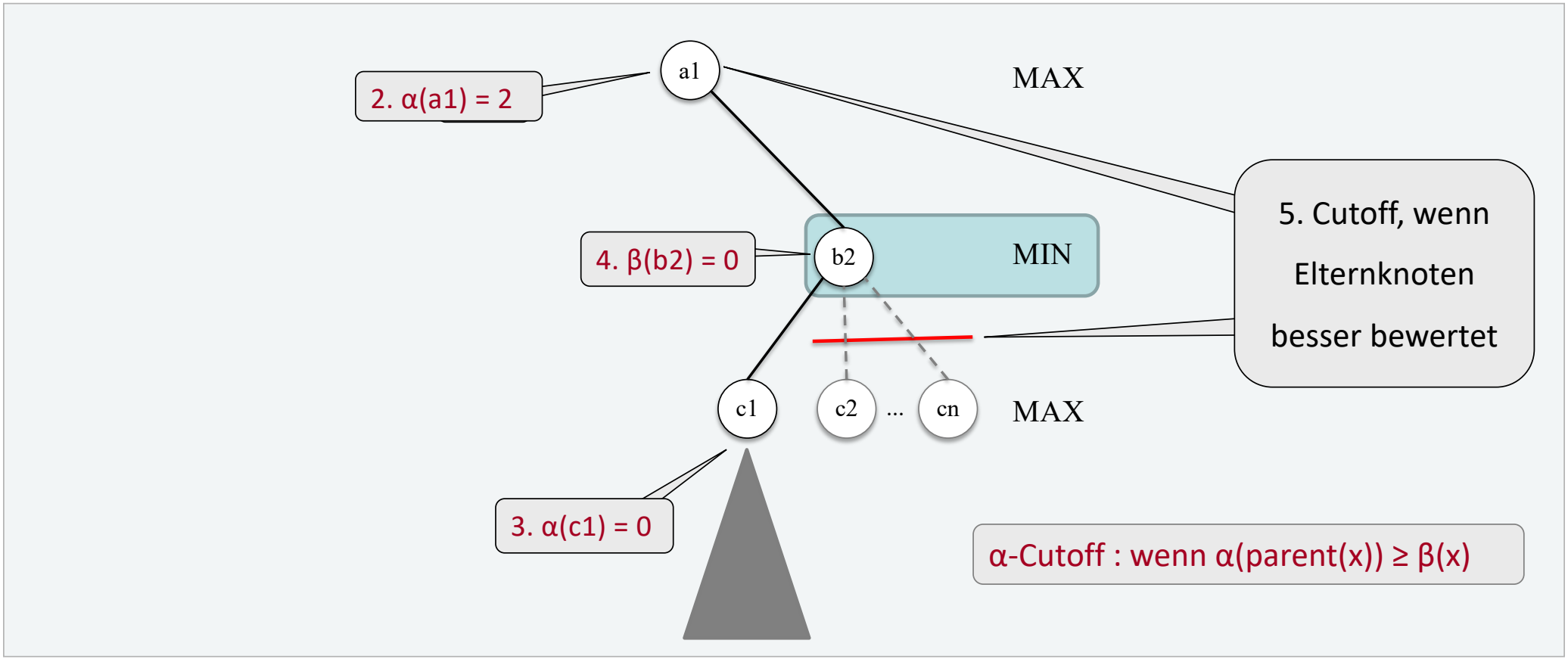


MAX

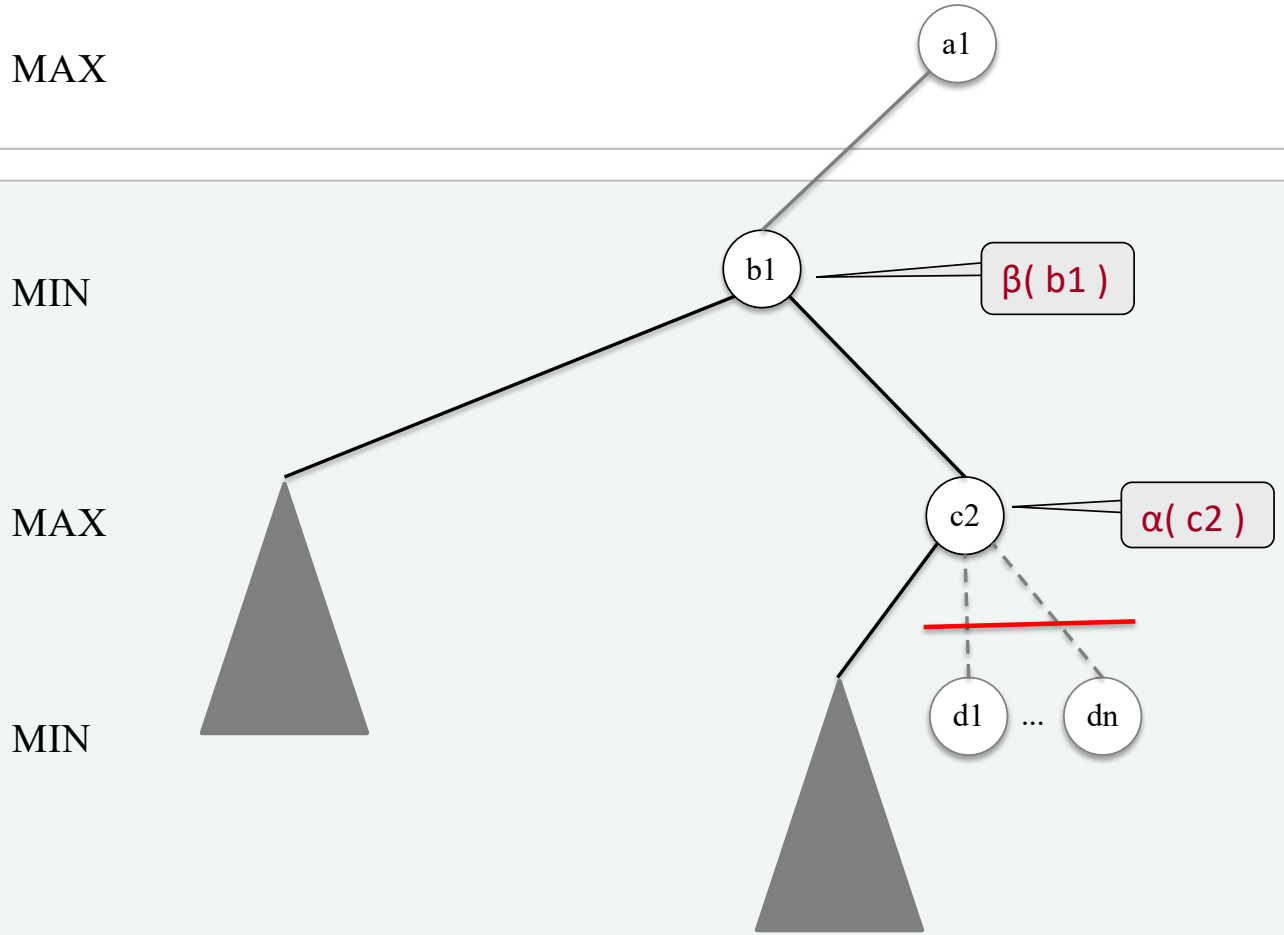
MIN

MAX

Irrationale Züge müssen nicht untersucht werden



β-Cutoff analog



β -Cutoff : wenn β(parent(x)) ≤ α(x)

α-β-Suche propagiert beste Zustandswerte durch den Spielbaum

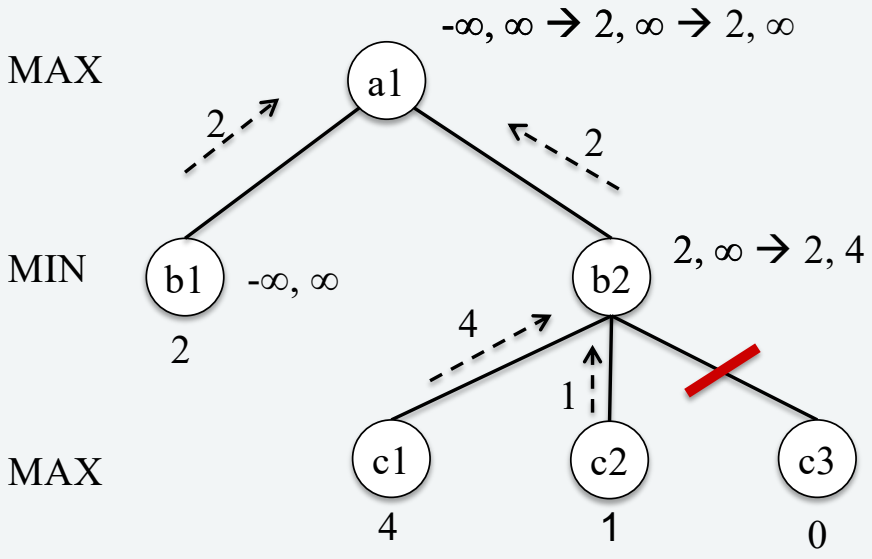
```
1.  Function alphaBetaMax( node, α, β, depth ):
2.      if depth = 0 or node is a terminal node
            then  return evaluate( node )
3.      foreach child of node do
4.          score := alphaBetaMin( child, α, β, depth - 1 )
5.          if  score ≥ β  then   return β
6.          if  score > α  then  α := score
7.      return α
```

```
1.  Function alphaBetaMin( node, α, β, depth ):
2.      if depth = 0 or node is a terminal node
            then  return evaluate( node )
3.      foreach child of node do
4.          score := alphaBetaMax( child, α, β, depth - 1 )
5.          if score ≤ α  then  return α
6.          if score < β  then   β := score
7.      return β
```

Aufruf:
alphaBetaMax(*wurzel*, -∞, ∞, *tiefenlimit*)

https://www.chessprogramming.org/Alpha-Beta#Max_versus_Min

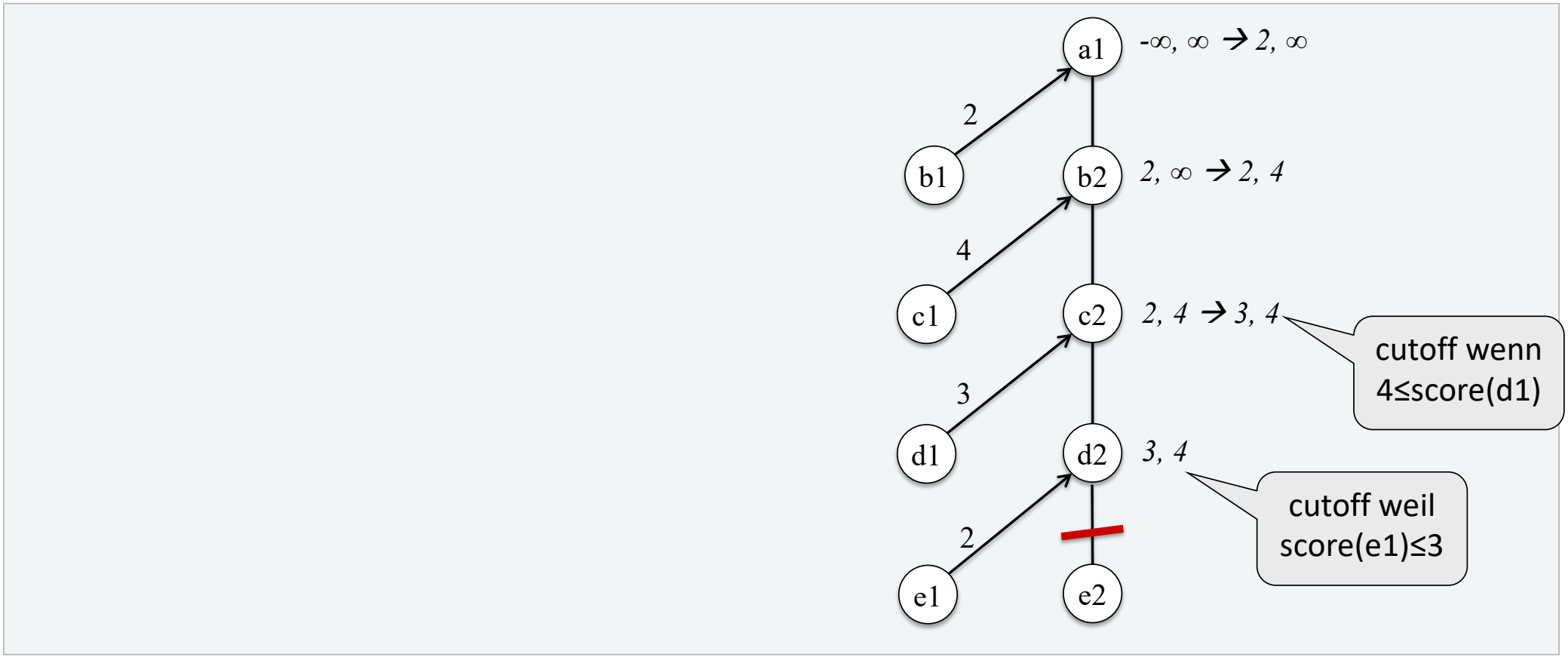
Anwendung des Alpha-Beta-Algorithmus



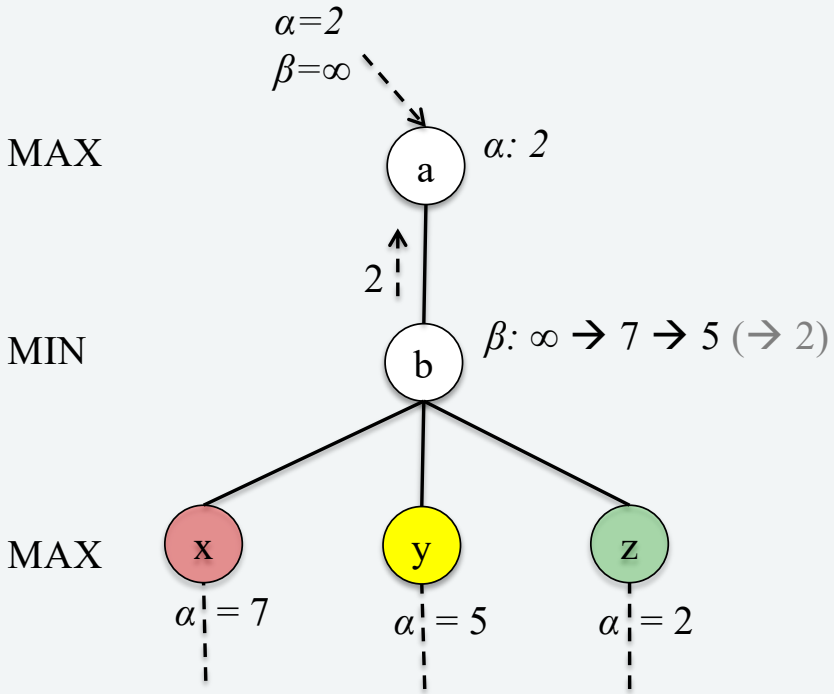

```
1.   Function alphaBetaMax( node, α, β, depth ):
2.     if depth = 0 or node is a terminal node
                then  return evaluate( node )
3.     foreach child of node do
4.       score := alphaBetaMin( child, α, β, depth - 1 )
5.       if  score ≥ β  then   return β
6.       if  score > α  then  α := score
7.     return α

8.   Function alphaBetaMin( node, α, β, depth ):
9.     if depth = 0 or node is a terminal node
                then  return evaluate( node )
10.    foreach child of node do
11.      score := alphaBetaMax( child, α, β, depth - 1 )
12.      if score ≤ α  then  return α
13.      if score < β  then   β := score
14.    return β
```

Auch tiefe Cutoffs sind möglich

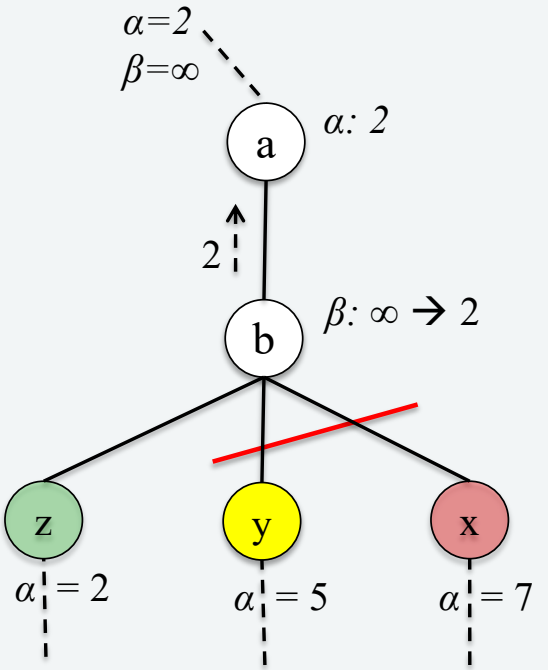


Durch eine gute Zugsortierung effektivere Cutoffs erzielen



(x, y und z werden besucht)

Idee: stärksten Zug zuerst untersuchen



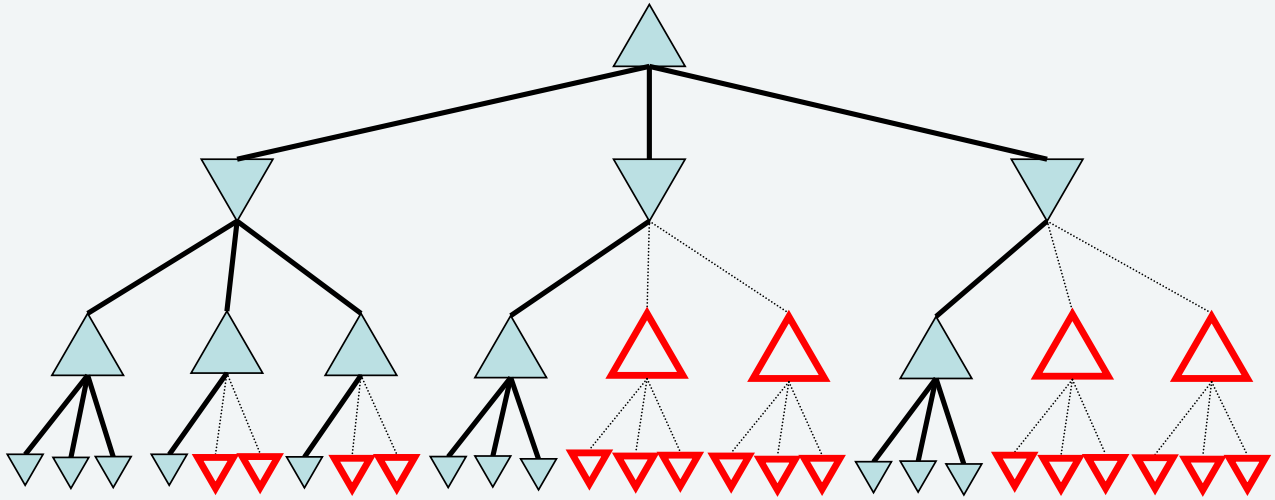
(maximale Beschneidung)

Doppelte Suchtiefe erreichbar mit optimaler Untersuchungsreihenfolge!

Zahl der Blätter in Ebene d (nach maximalen Cutoffs):

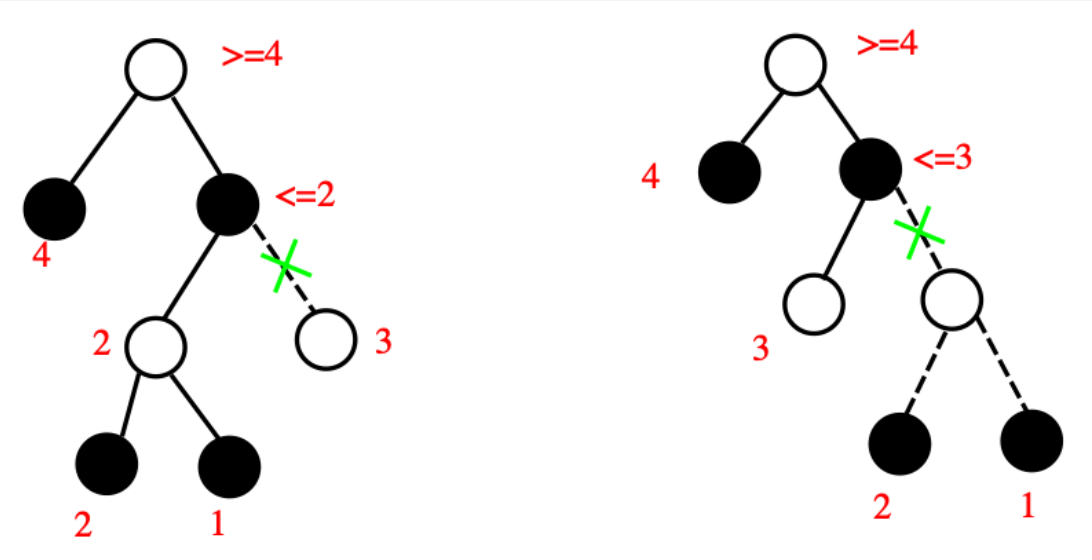
$$O\left(b^{d/2}\right) = O\left(\sqrt{b^d}\right)$$

nach [Knuth, Moore 1975]



50% Ersparnis maximal in diesem Spielbaum

Perfekte Zugsortierung ist nicht immer optimal

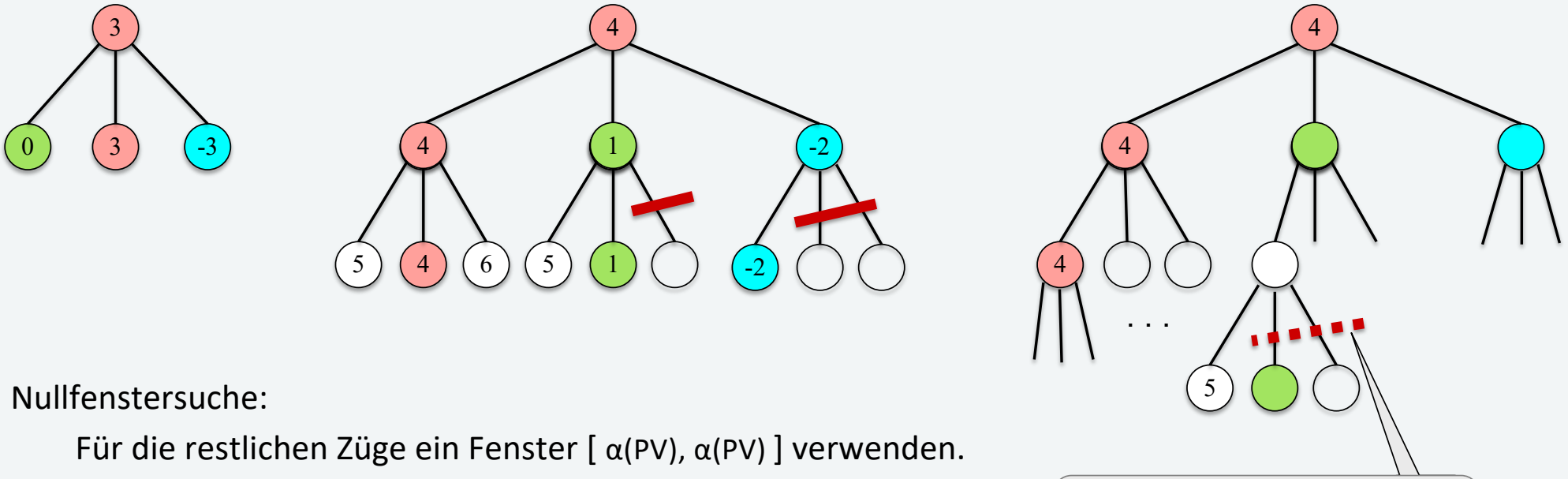


Beispiel aus: https://homepage.iis.sinica.edu.tw/~tshsu/tcg/2022/slides/slide6.pdf

Ausblick: Principal Variation für Zugsortierung verwenden

Verwendbar mit iterativer Tiefensuche:

Besten Pfad PV speichern und als Zugsortierung für nächste Iteration verwenden.



Nullfenstersuche:

Für die restlichen Züge ein Fenster [α(PV), α(PV)] verwenden.

Stellungsbewertung aus Transposition Table auslesen

1. d2-d4 d7-d5
2. c2-c4

1. c2-c4 d7-d5
2. de-d4

Stellung durch verschiedene Zugkombinationen erreichbar

Minimax würde denselben Teilbaum mehrfach durchsuchen

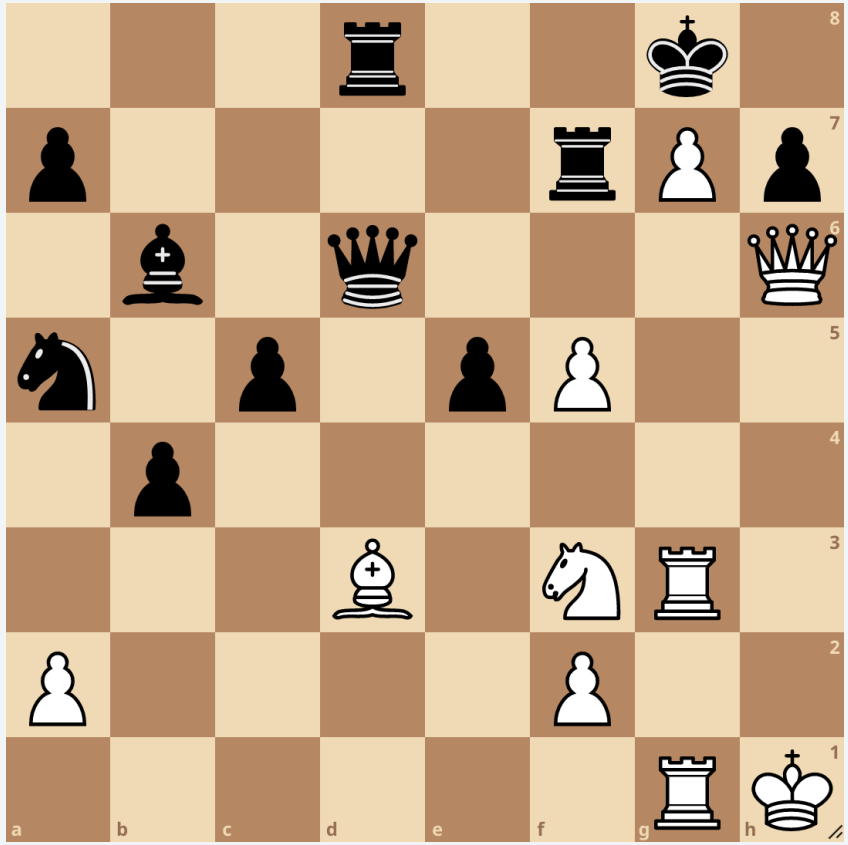


Transposition Table speichert bewertete Zustände → Redundanzvermeidung

Unruhige Stellungen sind extrem schwierig zu bewerten

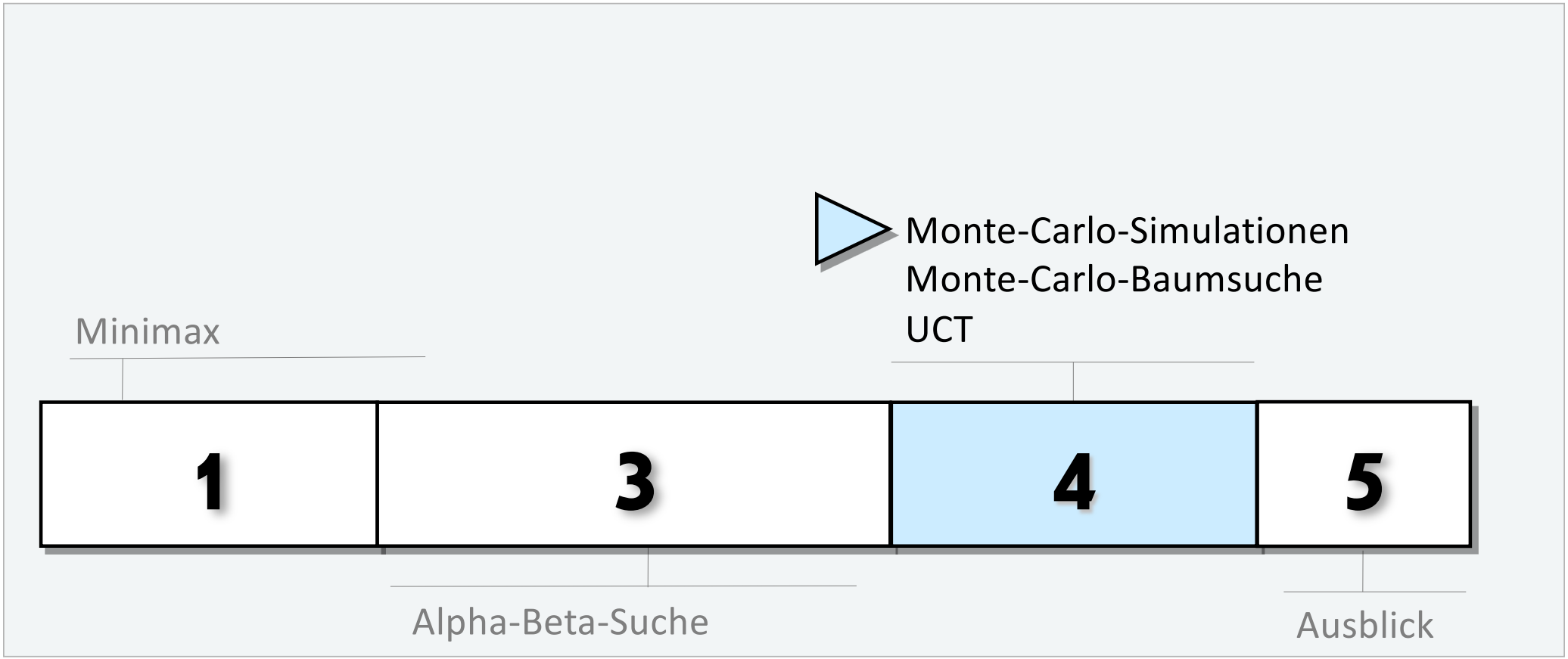
1. Dh6xh7+ Kg7xh7
2. g7-g8D+ Td8xg8
3. f5-f6+ Tg8-g6
4. Tg3-h3+ Kh7-g8
5. Tg1xg6+ Tf7-g7
6. Tg6xg7+ Kf8-f8
7. Th3-h8#

Ruhesuche: Schach- und Schlagzüge ausführen, bis ruhige Position erreicht, danach Bewertungsfunktion aufrufen



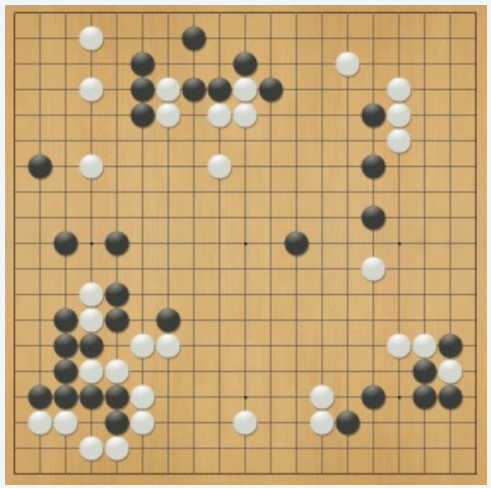
https://lichess.org/analysis/standard/3r2k1/p4rPp/1b1q3Q/n1p1pP2/1p6/3B1NR1/P4P2/6RK_w_-_-_0_1

Probabilistische Suche im Spielbaum



Limitierungen der Alpha-Beta-Suche

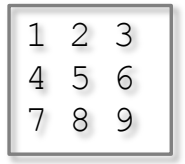
extrem hoher Verzweigungsgrad



http://gogui.sourceforge.net/

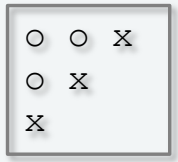
gute Bewertungs-funktion unbekannt oder teuer zu berechnen

Monte-Carlo-Simulationen sind zufallsgesteuerte Experimente

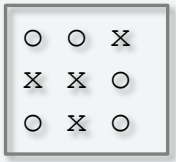


1. zahlreiche Spielsimulationen mit zufälligen Zügen
2. statistisch besten Zug ausführen

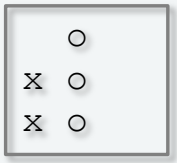
Gewinnwahrscheinlichkeit versus Rationalität



([1,3,2,5,4,6], -1)



([1,5,2,3,6,7,4,9], 0)



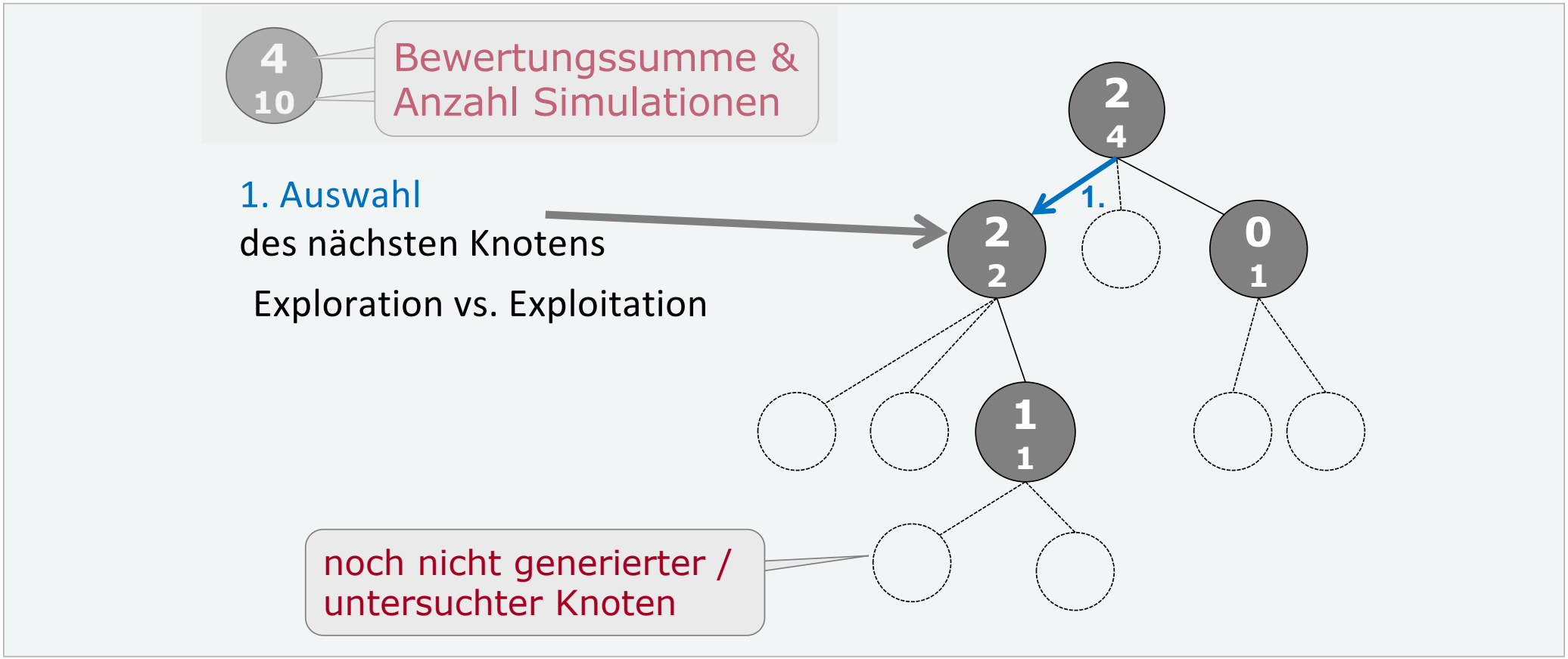
([2,4,5,7,8], +1)

…

Anytime-Eigenschaft

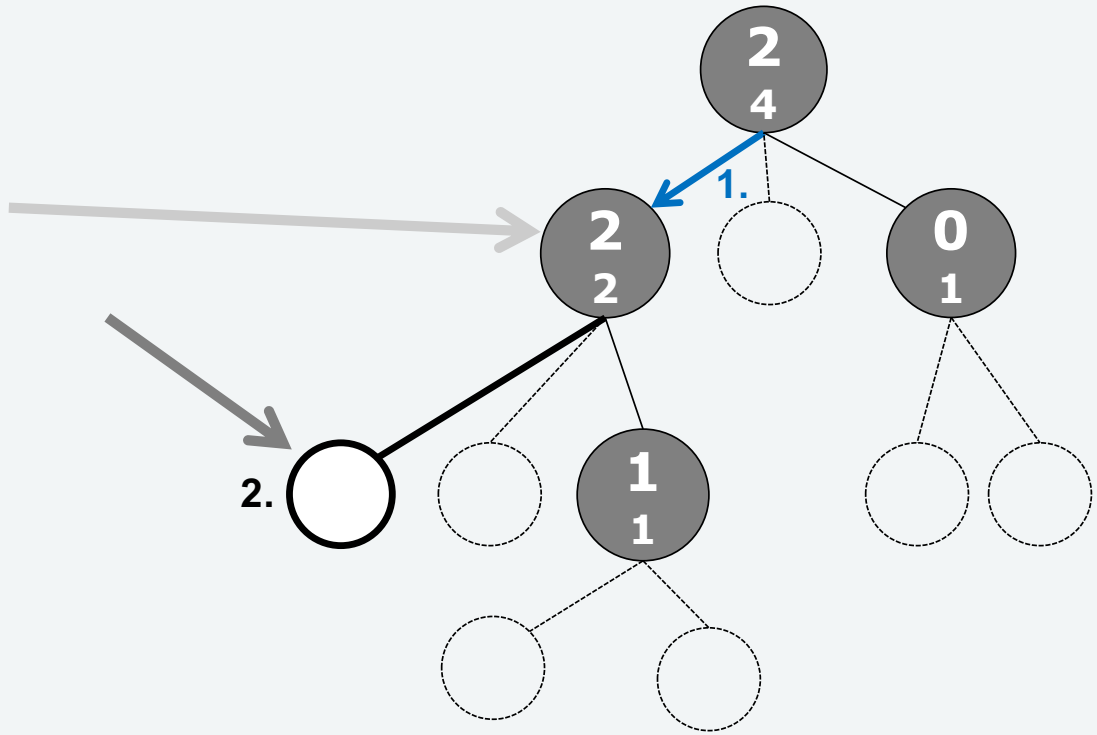
schnell

Monte-Carlo-Baumsuche (MCTS) realisiert eine heuristische Suche



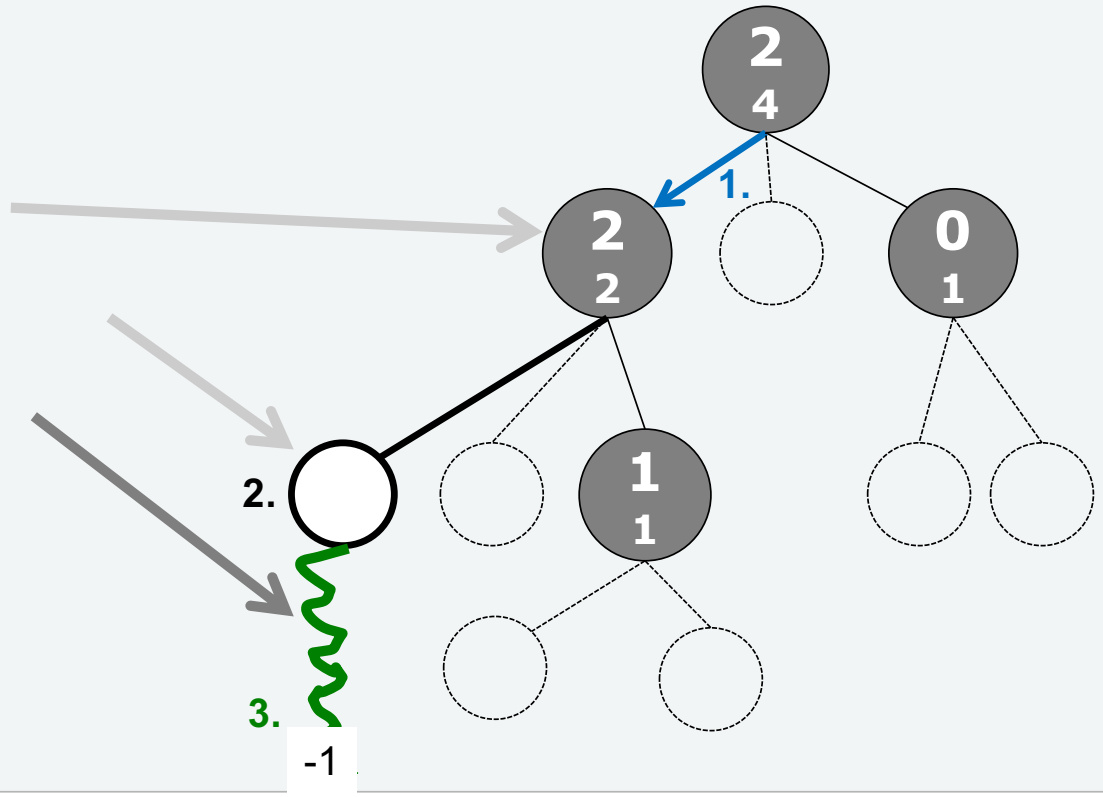
Ein Kindknoten wird generiert

1. Auswahl
2. Generierung



Der neue Blattknoten wird durch *eine* vollständige Simulation bewertet

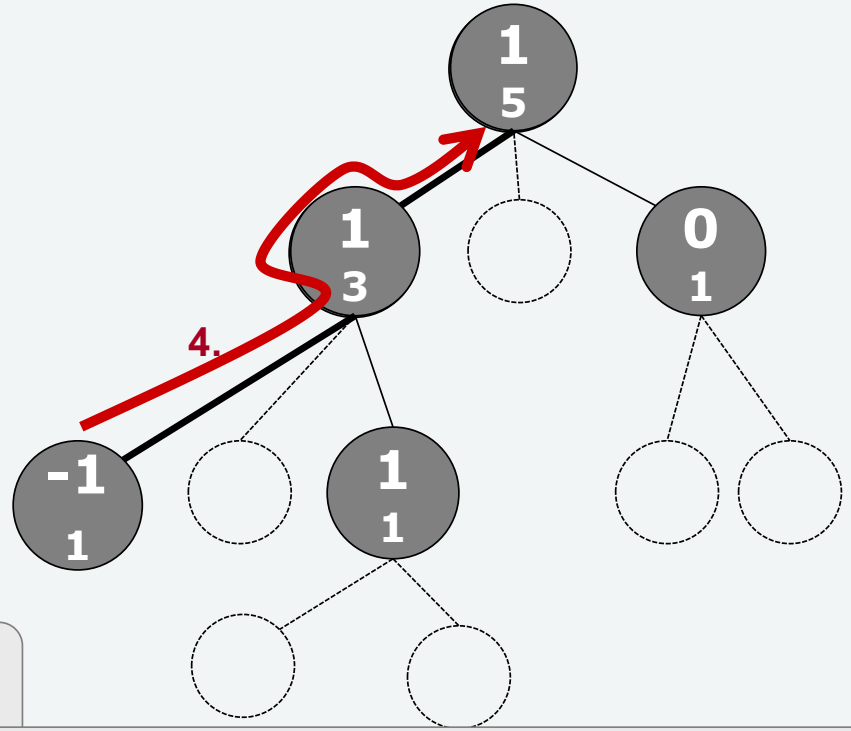
1. Auswahl
2. Generierung
3. Simulation



Durch Rückwärtspropagierung wird der Spielbaum aktualisiert

4. Schritt

1. Auswahl
2. Generierung
3. Simulation
4. Propagierung



nach Abschluss der Suche:
Wahl des besten Zuges

(Knoten mit den meisten Simulationen)

Zwischenfazit Monte Carlo-Baumsuche

speicherintensiv

Baum ist weiterverwendbar

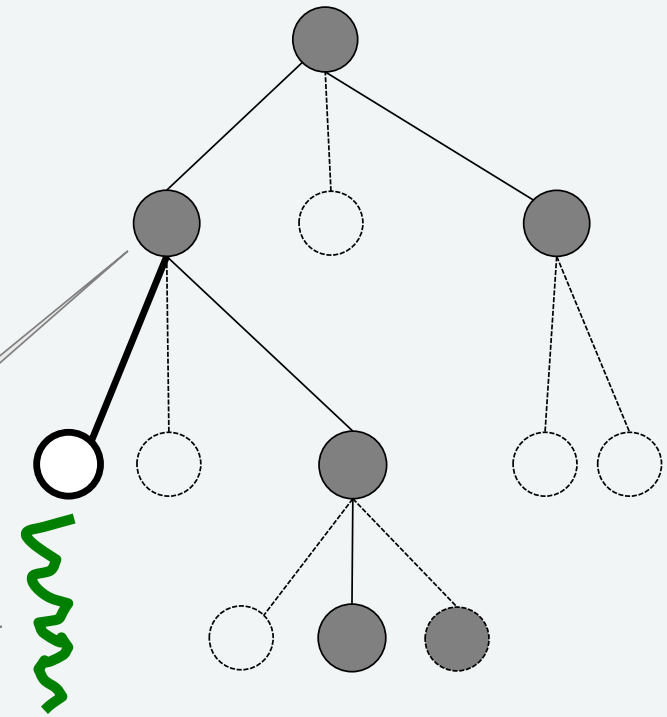
konvergiert zu Minimax-Werten

vermeidet Horizontproblem

schwach in taktischen Stellungen

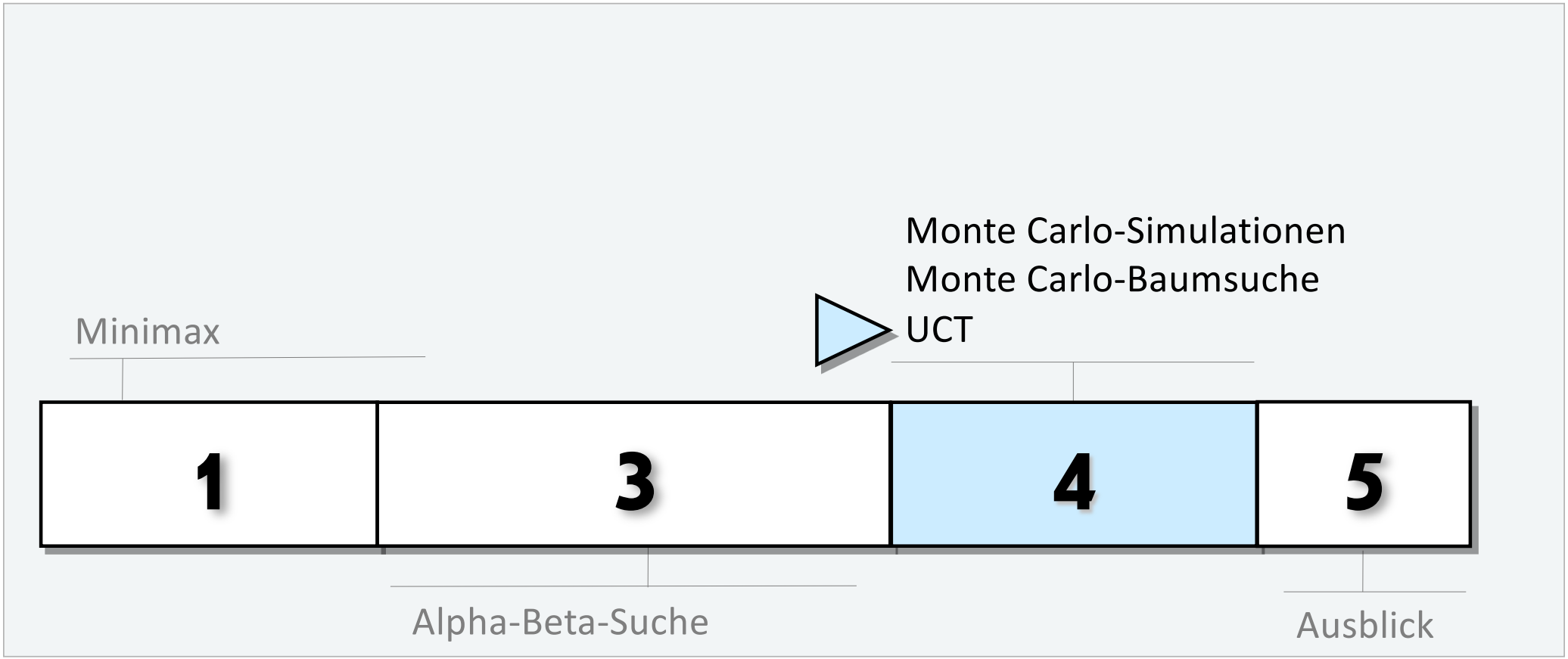
gut parallelisierbar

AlphaZero: Neuronale Netze + MCTS



Neuronales Netz
- bestimmt Kandidatenzüge
- ersetzt Simulation

Upper-Confidence Bounds Applied to Trees (UCT) lösen das Exploration-versus Exploitation-Problem



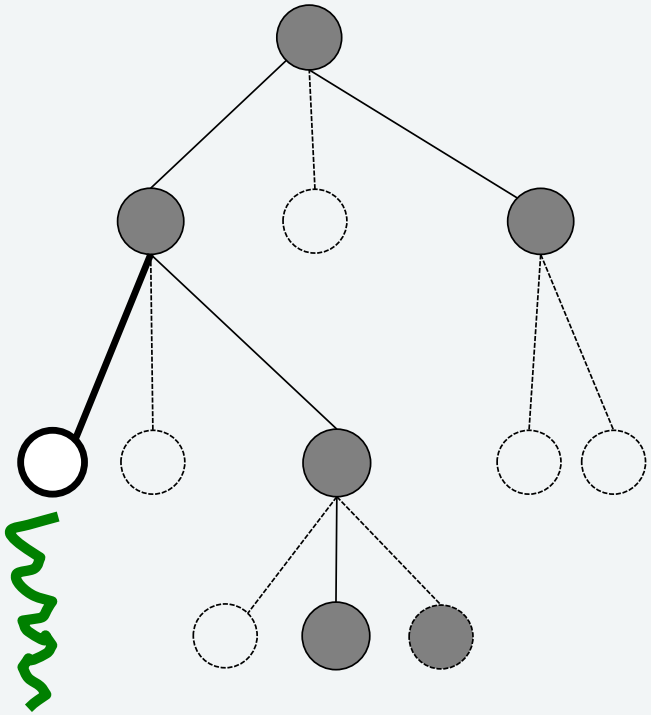
Baumtraversierung ist kritisch für die Qualität von MCTS-Entscheidungen

Exploitation

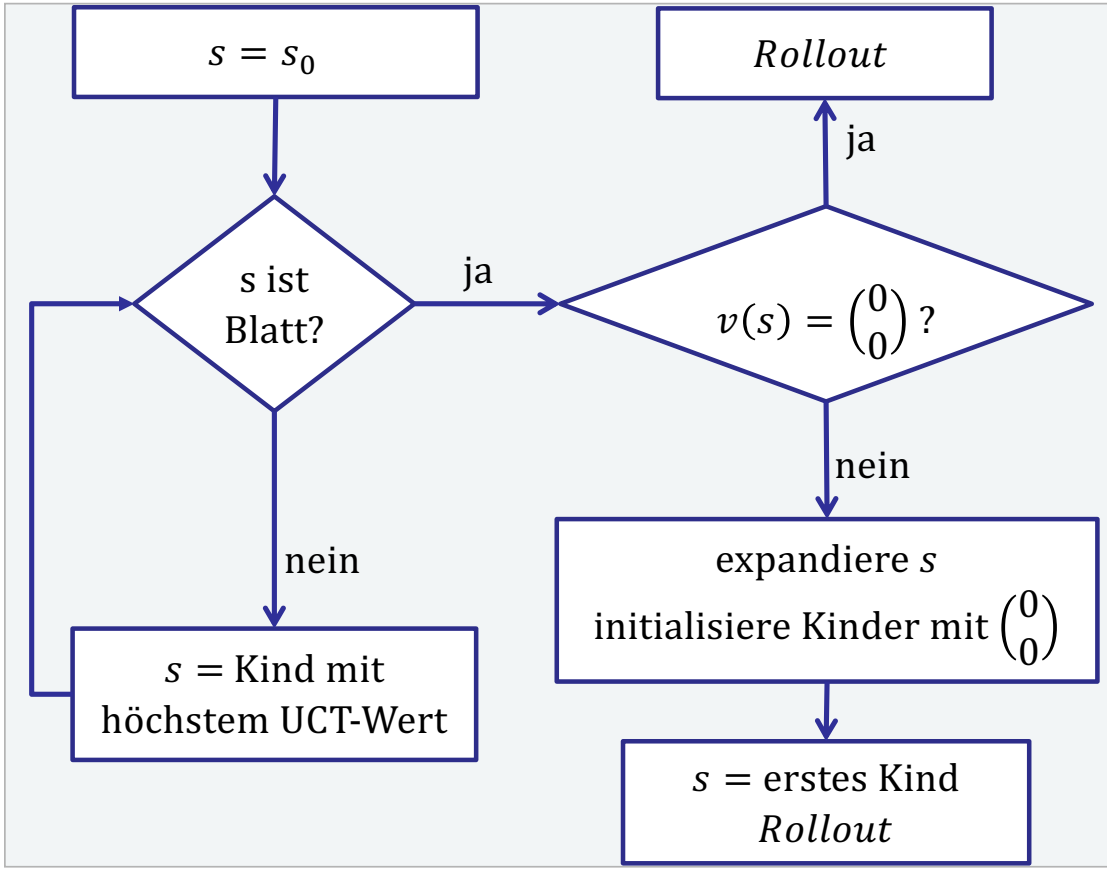
- vielversprechende Züge bevorzugt untersuchen
- generiert tiefe Bäume
- anfällig für widerlegbare Züge

Exploration

- noch nicht betrachtete Züge bevorzugt untersuchen
- entspricht Breitensuche
- schwächer als Alpha-Beta-Suche



Integration von UCT in den Auswahlprozess



$$UCT_i = x_i + C \times \sqrt{\frac{ln(N)}{n_i}}$$

x_i : Gewinnrate des Knotens i

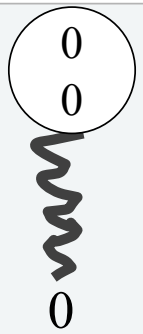
n_i : Zahl der Simulationen in i

N : Zahl der Simulationen im Elternknoten

C : Explorationskonstante (C > 0)

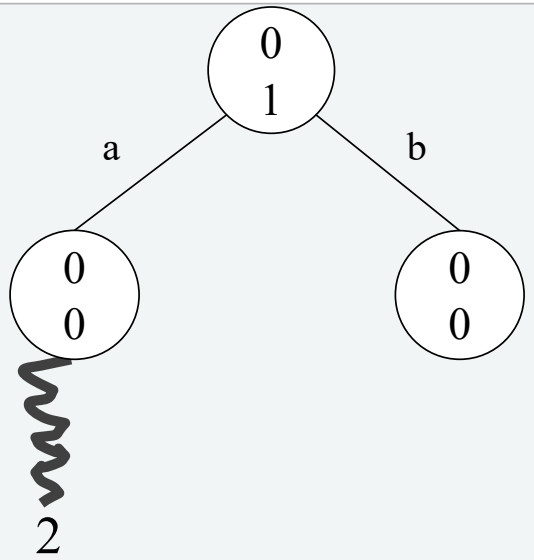
Beispiel mit C=2

$$UCT_i = x_i + C \times \sqrt{\frac{ln(N)}{n_i}}$$



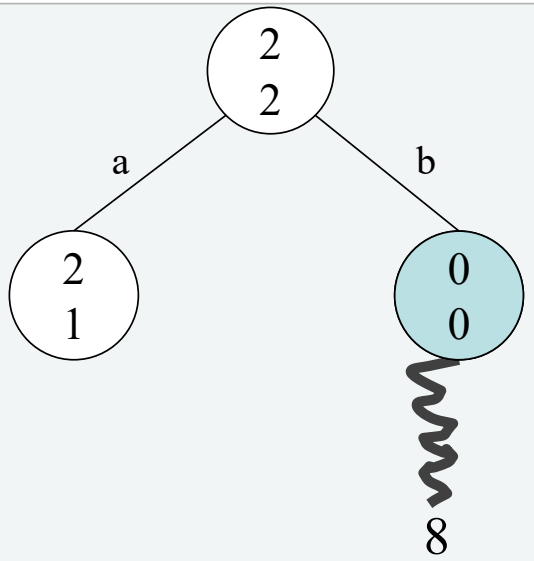
Beispiel mit C=2

$$UCT_i = x_i + C \times \sqrt{\frac{ln(N)}{n_i}}$$



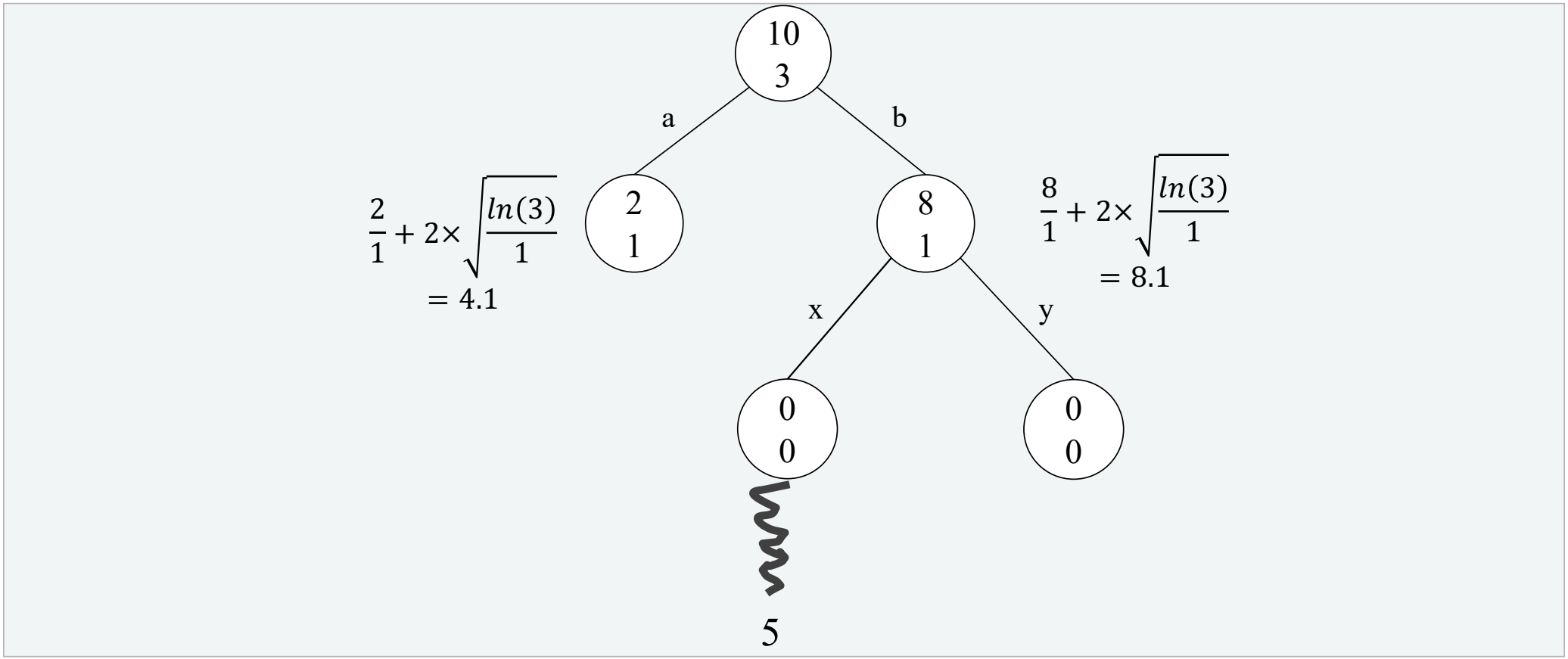
Beispiel mit C=2

$$UCT_i = x_i + C \times \sqrt{\frac{ln(N)}{n_i}}$$



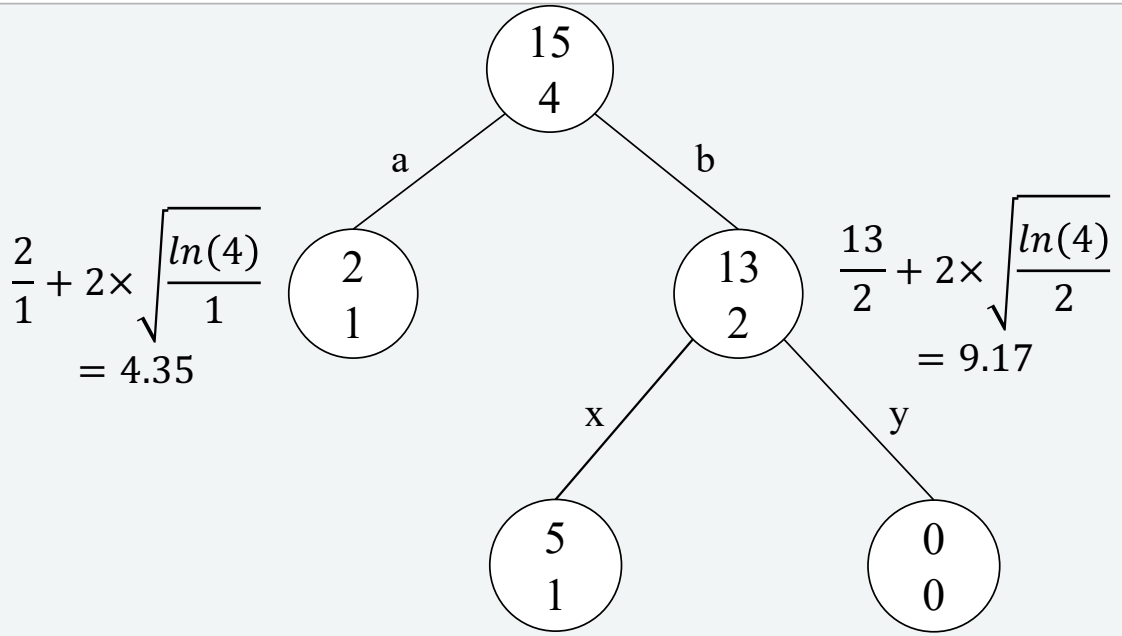
Beispiel mit C=2

$$UCT_i = x_i + C \times \sqrt{\frac{ln(N)}{n_i}}$$



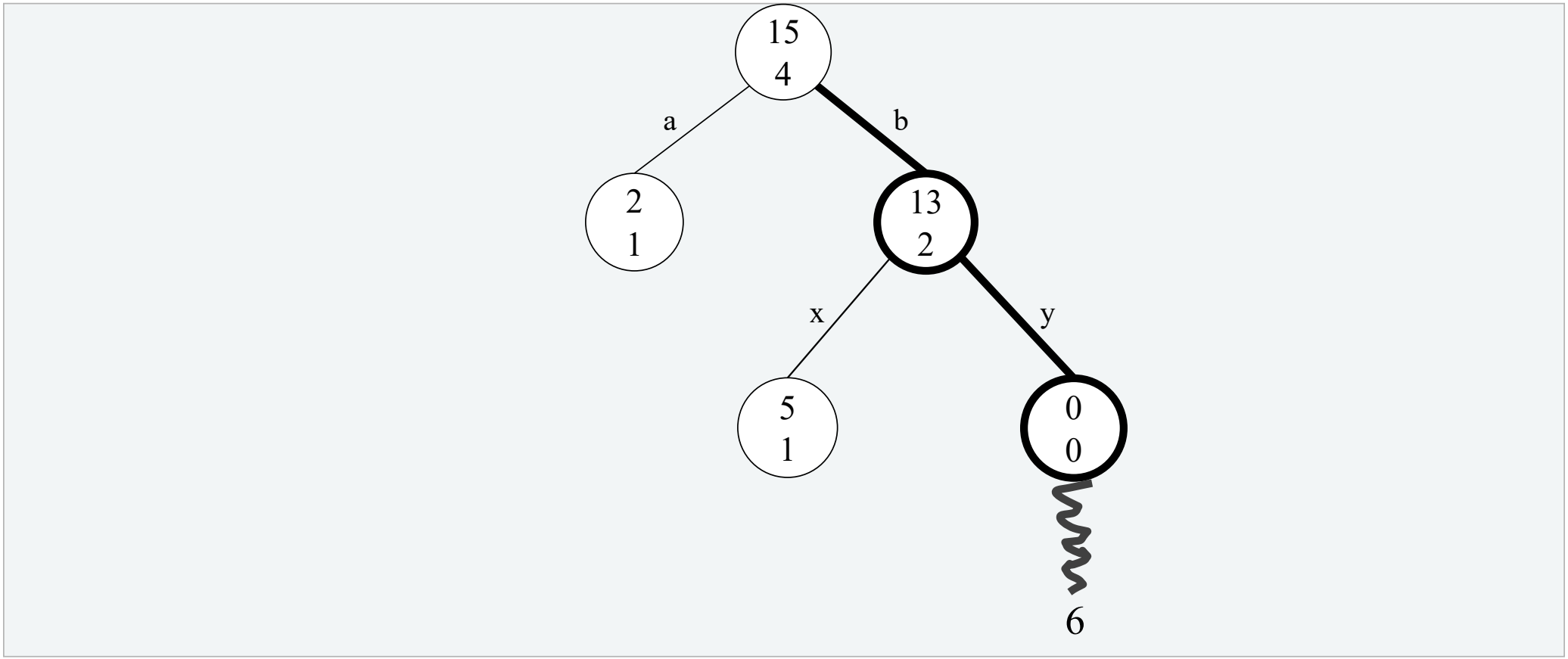
Beispiel mit C=2

$$UCT_i = x_i + C \times \sqrt{\frac{ln(N)}{n_i}}$$



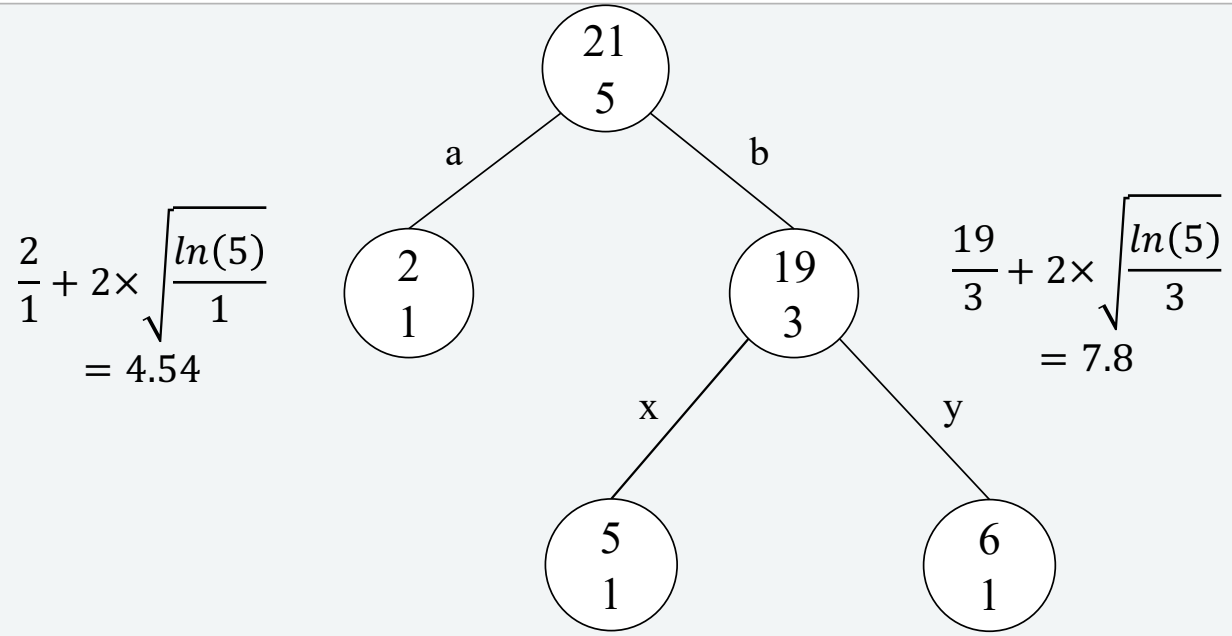
Beispiel mit C=2

$$UCT_i = x_i + C \times \sqrt{\frac{ln(N)}{n_i}}$$



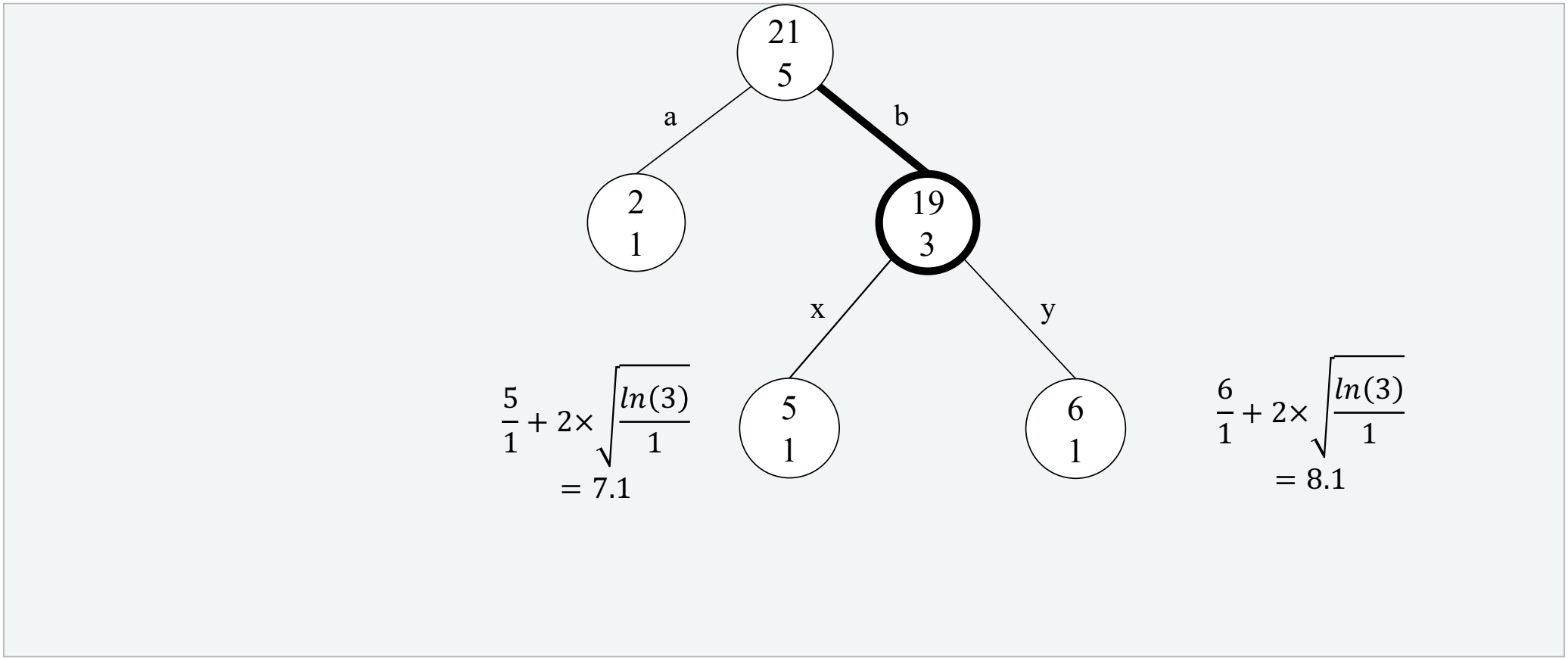
Beispiel mit C=2

$$UCT_i = x_i + C \times \sqrt{\frac{ln(N)}{n_i}}$$



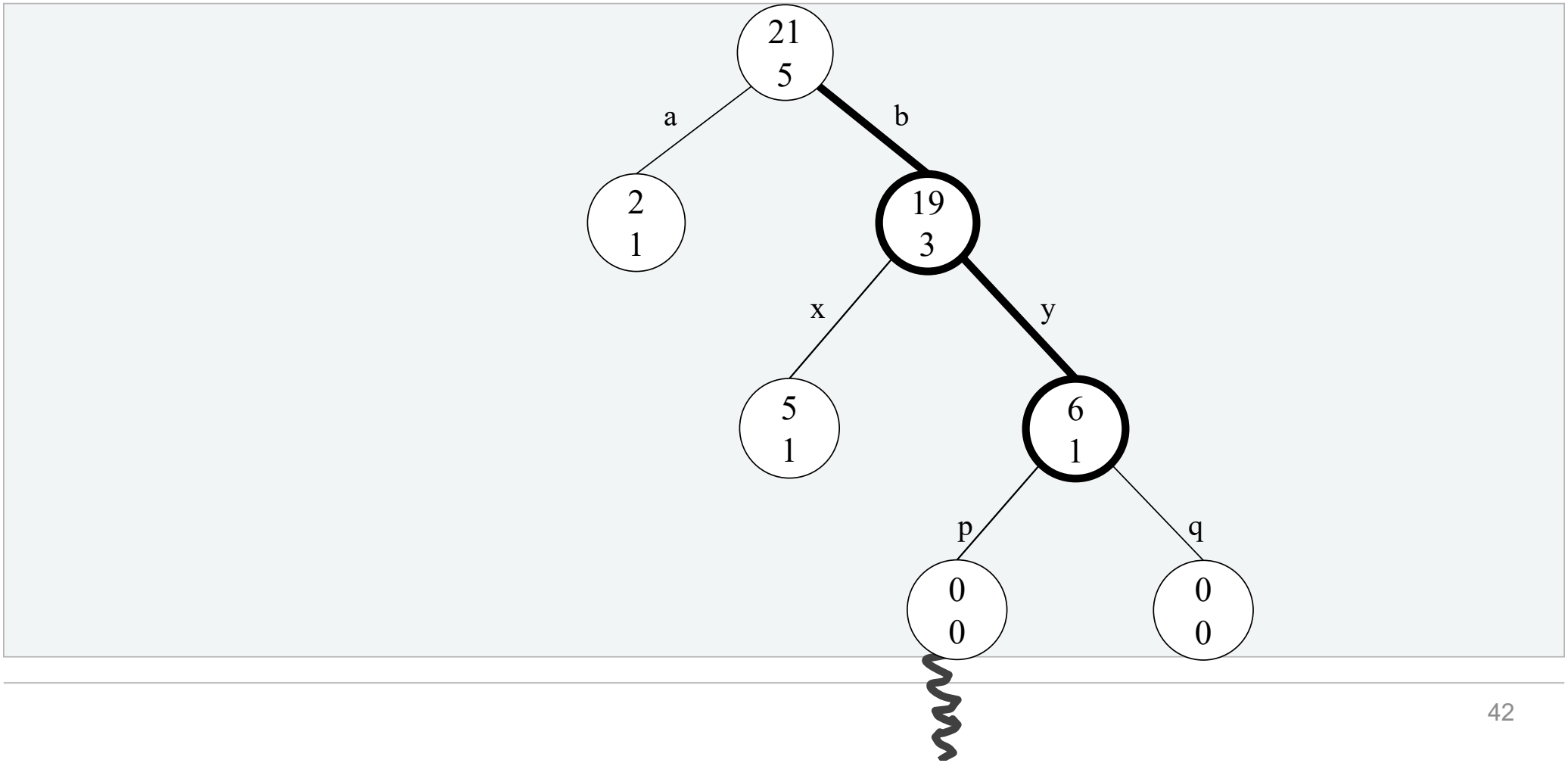
Beispiel mit C=2

$$UCT_i = x_i + C \times \sqrt{\frac{ln(N)}{n_i}}$$



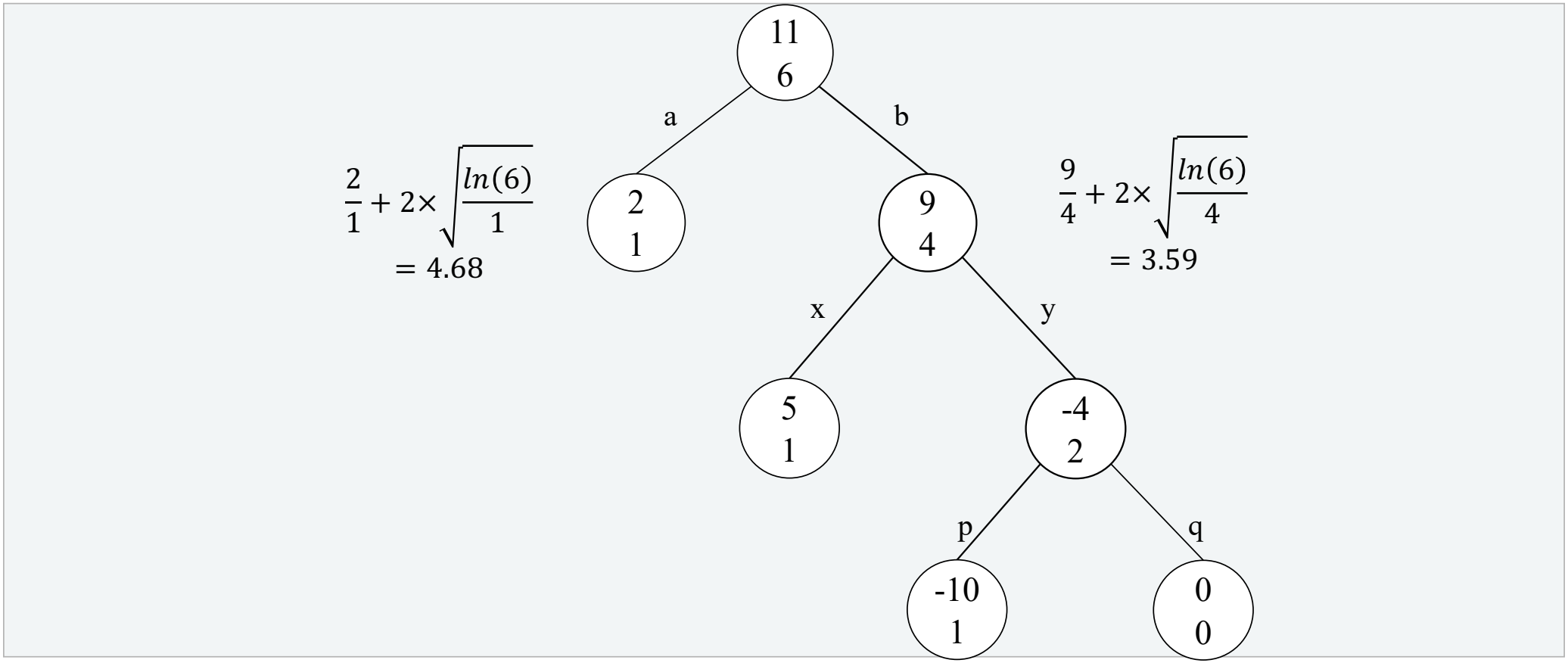
Beispiel mit C=2

$$UCT_i = x_i + C \times \sqrt{\frac{ln(N)}{n_i}}$$

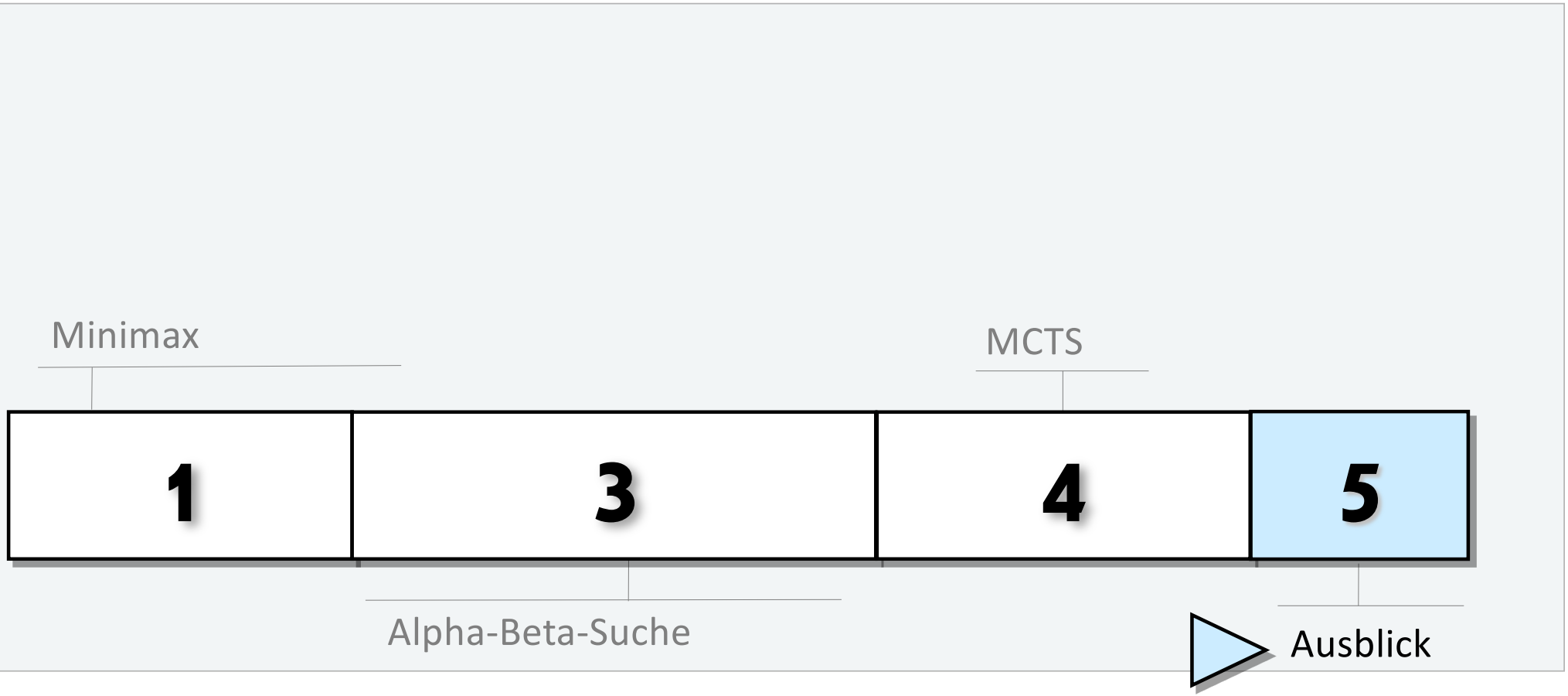


Hoher negativer Rollout für p zwingt MCTS in die Untersuchung von a

$$UCT_i = x_i + C \times \sqrt{\frac{ln(N)}{n_i}}$$



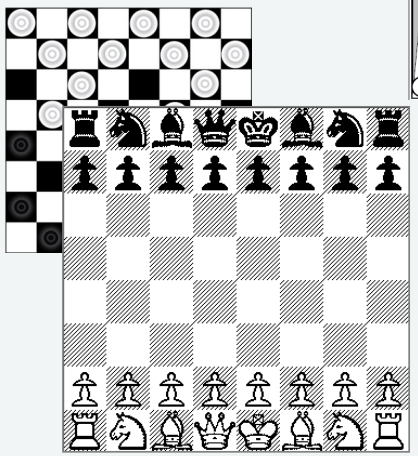
Agenda strategische Spiele

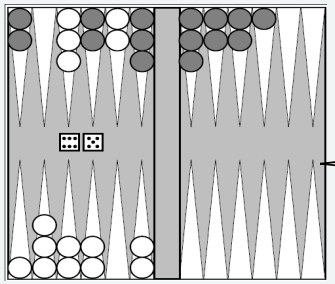


Wo stehen wir heute?

Übermenschliche Spielstärke

noch nicht meisterlich





erlernte Bewertungs-funktion



MCTS + neuronales Netz für Kandidatenzüge

Echtzeitstrategiespiele

Teamarbeit

bluffen, adaptieren



Deep Blue: ∅ 12 Halbzüge (max. 40). 200 Millionen Positionen / Sek.



Dr.-Ing. Stefan Fricke

TU Berlin | Fachgebiet *Service-centric Networking*
Ernst-Reuter-Platz 1 | 10587 Berlin | Germany

stefan.fricke@tu-berlin.de

https://twitter.com/kuepp

https://www.xing.com/in/axelkuepper/

http://www.snet.tu-berlin.de/kuepper

